

# 人工智能期末复习课详细提纲

## Contents

|          |                           |           |
|----------|---------------------------|-----------|
| <b>1</b> | <b>复习课总目标</b>             | <b>2</b>  |
| <b>2</b> | <b>残差网络基础</b>             | <b>3</b>  |
| 2.1      | 残差网络 ResNet               | 3         |
| 2.2      | 思考题                       | 3         |
| <b>3</b> | <b>卷积神经网络 CNN</b>         | <b>3</b>  |
| 3.1      | 图像数据的特征：关联性               | 3         |
| 3.2      | 图像数据的特征：不变性               | 4         |
| 3.3      | 大脑如何处理图像：类视皮层设计           | 4         |
| 3.4      | 卷积运算                      | 4         |
| 3.5      | Zero Padding              | 5         |
| 3.6      | Stride                    | 6         |
| 3.7      | 多通道卷积                     | 7         |
| 3.8      | 卷积层尺寸计算总结                 | 8         |
| 3.9      | 池化 Pooling                | 9         |
| 3.10     | CNN 整体结构与层的组合             | 10        |
| 3.11     | 关键记忆                      | 10        |
| 3.12     | 思考题                       | 11        |
| <b>4</b> | <b>循环神经网络 RNN 与 LSTM</b>  | <b>11</b> |
| 4.1      | 自然语言处理 NLP 的任务特点          | 11        |
| 4.2      | 数据准备                      | 12        |
| 4.3      | 分词、one-hot 与 embedding    | 12        |
| 4.4      | Next Token Prediction     | 13        |
| 4.5      | RNN 的设计                   | 14        |
| 4.6      | BPTT 与梯度问题                | 15        |
| 4.7      | LSTM                      | 16        |
| 4.8      | 关键记忆                      | 17        |
| 4.9      | 思考题                       | 17        |
| <b>5</b> | <b>Transformer</b>        | <b>17</b> |
| 5.1      | 为什么需要 Transformer         | 17        |
| 5.2      | Embedding 与位置编码           | 17        |
| 5.3      | 注意力机制直观理解                 | 17        |
| 5.4      | Q、K、V                     | 18        |
| 5.5      | Causal Mask               | 18        |
| 5.6      | Transformer Block、FNN 与输出 | 18        |
| 5.7      | 关键记忆                      | 19        |
| 5.8      | 思考题                       | 19        |
| <b>6</b> | <b>神经网络训练现象：频率原则</b>      | <b>19</b> |
| 6.1      | 过参数化与隐式偏好                 | 19        |
| 6.2      | 频率、低频与高频                  | 19        |

|           |                       |           |
|-----------|-----------------------|-----------|
| 6.3       | 频率原则                  | 20        |
| 6.4       | 早停 Early stopping     | 20        |
| 6.5       | 思考题                   | 20        |
| <b>7</b>  | <b>参数凝聚与解的平坦性</b>     | <b>21</b> |
| 7.1       | 初始化大小的影响              | 21        |
| 7.2       | 参数凝聚                  | 21        |
| 7.3       | 平坦解与尖锐解               | 21        |
| 7.4       | 思考题                   | 21        |
| <b>8</b>  | <b>大模型介绍</b>          | <b>22</b> |
| 8.1       | 什么是大模型                | 22        |
| 8.2       | Encoder、Decoder 与常见架构 | 22        |
| 8.3       | 预训练、微调、提示词、蒸馏         | 23        |
| 8.4       | 大模型中的现象               | 23        |
| 8.5       | 思考题                   | 23        |
| <b>9</b>  | <b>强化学习</b>           | <b>23</b> |
| 9.1       | 强化学习是什么               | 23        |
| 9.2       | 状态、观察、动作空间、奖励         | 24        |
| 9.3       | 回报与折扣因子               | 24        |
| 9.4       | 情节型任务与连续型任务           | 25        |
| 9.5       | 探索与利用                 | 25        |
| 9.6       | 策略、价值函数与最优策略          | 25        |
| 9.7       | MDP 与马尔可夫性            | 26        |
| 9.8       | Bellman 方程的直观含义       | 26        |
| 9.9       | DP、MC 与深度强化学习         | 26        |
| 9.10      | 强化学习应用                | 27        |
| 9.11      | 思考题                   | 27        |
| <b>10</b> | <b>最后一遍串讲清单</b>       | <b>27</b> |
| 10.1      | 残差网络必会                | 27        |
| 10.2      | CNN 必会                | 28        |
| 10.3      | RNN/LSTM 必会           | 28        |
| 10.4      | Transformer 必会        | 28        |
| 10.5      | 训练现象必会                | 28        |
| 10.6      | 大模型必会                 | 28        |
| 10.7      | 强化学习必会                | 29        |
| <b>11</b> | <b>不建议作为考试重点的内容</b>   | <b>29</b> |
| <b>12</b> | <b>两节复习课建议讲法</b>      | <b>29</b> |
| 12.1      | 第一节课：网络结构主线           | 29        |
| 12.2      | 第二节课：训练现象、大模型、强化学习    | 30        |

适用对象：大一下学生。

适用题型：选择题、判断题、填空题。

复习定位：这份提纲不是题库，而是复习课讲授顺序。提纲采用“章级大知识点 -> 小知识点 -> 关键记忆 -> 思考题”的层次组织。

考试边界：从卷积神经网络开始到强化学习为主；第 11-12 节只保留残差网络相关基础。课件中标记为“阅读材料”的内容不作为复习重点。

## 1 复习课总目标

学生复习完后，应当能做到：

1. 看到一个神经网络结构名，能说出它解决什么类型的问题。

2. 看到一个模块名，能说出它在网络中的作用。
3. 看到一个容易混淆的说法，能判断对错并说出原因。
4. 对 CNN 能完成最简单的输出尺寸、通道数、参数量计算。
5. 对强化学习能说清状态、动作、奖励、策略、价值函数、回报、MDP 的含义。
6. 对大模型能说清预训练、微调、上下文学习、思维链、幻觉等基本概念。

## 2 残差网络基础

第 11-12 节中，全连接神经网络的细节不作为复习重点；这里只保留残差网络相关内容，作为理解深层网络训练困难的基础。

### 2.1 残差网络 ResNet

复习重点：

1. 深层普通网络不一定比浅层网络好，可能出现退化问题。
2. 退化问题指网络加深后训练误差和测试误差反而变差。
3. 深层网络还可能遇到梯度消失或梯度爆炸。
4. 残差连接让网络学习  $F(x) + x$ 。
5. 如果额外层学不到有用特征，至少可以更容易近似恒等映射。
6. 残差连接帮助信息和梯度在深层网络中传播。
7. 残差网络的核心不是“堆更多层一定更好”，而是让深层网络更容易训练。

关键表述：

普通层：输出 =  $F(x)$

残差层：输出 =  $F(x) + x$

易错点：

1. 残差连接不是简单地让网络“变浅”，而是让网络更容易学习恒等映射和优化深层结构。
2. 残差连接本身不一定显著增加参数量。
3. ResNet 不是只用于解决过拟合，它主要缓解深层网络训练困难和退化问题。
4. 深层普通网络表现变差不一定是因为表达能力不足，也可能是优化更困难。

### 2.2 思考题

1. 判断：残差连接的形式可以写成  $F(x) + x$ 。  
答案：正确。
2. 判断：普通深层网络训练效果变差，一定是因为网络表达能力不够。  
答案：错误。也可能是优化更困难、梯度传播更困难。
3. 填空：残差网络希望让额外层更容易学习一种特殊映射，叫做 \_\_\_\_\_ 映射。  
答案：恒等。
4. 选择：ResNet 主要缓解的问题是：A. 图像颜色太多 B. 深层网络训练困难 C. 数据集文件太大 D. 类别数太少。  
答案：B。
5. 判断：残差连接本身的核心作用是让信息和梯度更容易在深层网络中传播。  
答案：正确。

## 3 卷积神经网络 CNN

这一章按 PDF 顺序复习：图像特性 -> 类视皮层启发 -> 卷积计算 -> padding -> stride -> 多通道卷积 -> 池化 -> CNN 结构。页码均指本章 PDF 页码。

### 3.1 图像数据的特征：关联性

页码：p3-p5

图像有空间结构。相邻像素通常更相关，距离越远，关联通常越弱。

图像局部关联性：图像中距离较近的像素或局部区域通常更相关，距离越远，相关性通常越弱。CNN 的局部连接设计正是为了利用这种特点。

| 模型    | 形式                  | 记忆           |
|-------|---------------------|--------------|
| 指数衰减  | $y = e^{-kx}$       | 下降较快         |
| 幂级数衰减 | $y = \frac{1}{x^k}$ | 下降较慢，长程关联更明显 |

幂级数衰减可以看作许多指数衰减成分的叠加：

$$\int_0^{\infty} e^{-\alpha t} d\alpha = \frac{1}{t}$$

### 3.2 图像数据的特征：不变性

页码：p6

图像识别不应只记住像素位置。物体轻微平移、局部扰动或光照稍变时，类别通常不应改变。

不变性：输入发生小幅平移、扰动或光照变化时，模型仍应尽量识别出相同语义类别。

### 3.3 大脑如何处理图像：类视皮层设计

页码：p7-p10

图像处理可以理解为：先提取局部简单特征，再逐步整合成复杂信息。

| 类比对象 | 作用          | CNN 中的对应模块  |
|------|-------------|-------------|
| 简单细胞 | 对位置、边缘、方向敏感 | 卷积层         |
| 复杂细胞 | 整合多个局部输入    | 池化、全连接等后续模块 |

| CNN 模块 | 作用      |
|--------|---------|
| 卷积层    | 提取局部特征  |
| 池化层    | 汇总局部信息  |
| 全连接层   | 综合特征    |
| 输出层    | 输出类别或概率 |

### 3.4 卷积运算

页码：p11-p13

卷积核是一个小窗口，例如  $3 \times 3$ 。它在图像上滑动，每次覆盖一个局部区域，把局部区域和卷积核对应位置相乘后求和。

卷积运算：卷积核在输入图像或特征图上滑动，每次对局部窗口做对应位置相乘再求和，得到输出特征图中的一个值。

# 卷积运算



右边的例子中用的是  $g(-u, -v)$ , 是否等价?

是等价, 本质上, 做个变量代换即可。

$$F(x, y) = \sum_{u, v} f(x - u, y - v)g(u, v)$$

$$F(x, y) = \int f(x - u, y - v)g(u, v)dudv$$

卷积核

|             |            |            |
|-------------|------------|------------|
| $g(-1, 1)$  | $g(0, 1)$  | $g(1, 1)$  |
| $g(-1, 0)$  | $g(0, 0)$  | $g(1, 0)$  |
| $g(-1, -1)$ | $g(0, -1)$ | $g(1, -1)$ |

图像

|  |              |            |              |  |
|--|--------------|------------|--------------|--|
|  |              |            |              |  |
|  | $(x-1, y+1)$ | $(x, y+1)$ | $(x+1, y+1)$ |  |
|  | $(x-1, y)$   | $(x, y)$   | $(x+1, y)$   |  |
|  | $(x-1, y-1)$ | $(x, y-1)$ | $(x+1, y-1)$ |  |
|  |              |            |              |  |

$$F(x, y) = f(x-1, y+1)g(-1, 1) + f(x, y+1)g(0, 1) + f(x+1, y+1)g(1, 1) \\ + f(x-1, y)g(-1, 0) + f(x, y)g(0, 0) + f(x+1, y)g(1, 0) \\ + f(x-1, y-1)g(-1, -1) + f(x, y-1)g(0, -1) + f(x+1, y-1)g(1, -1)$$

[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



Figure 1: p11 卷积核在图像局部区域上做加权求和

## 3.4.1 单通道卷积示意

图像局部区域  $x$

卷积核  $K$

1 2 0  
3 1 2  
0 1 1

1 0 -1  
1 0 -1  
1 0 -1

一次卷积输出 =

$$1 \times 1 + 2 \times 0 + 0 \times (-1) \\ + 3 \times 1 + 1 \times 0 + 2 \times (-1) \\ + 0 \times 1 + 1 \times 0 + 1 \times (-1) \\ = 1$$

卷积核继续向右或向下滑动, 就得到下一个位置的输出。所有位置的输出组成新的特征图。

| 模型  | 连接方式           | 对图像结构的利用   |
|-----|----------------|------------|
| MLP | 常把图像展平成向量, 全连接 | 容易弱化二维空间结构 |
| CNN | 局部连接, 卷积核滑动    | 直接利用局部关联   |

## 3.5 Zero Padding

页码: p14

Padding 是在图像边缘补值, 常见是补 0。作用是控制卷积后图像大小, 避免图像尺寸过快变小。

**Padding:** Padding 是在输入边缘补值, 常见是补 0, 用来控制卷积后的空间尺寸, 并让边缘位置也能参与更多卷积计算。

### 3.5.1 Padding 示意

## Zero Padding——解决图像过卷积后大小改变的问题



Padding=0

Padding=1

Padding=2

原图像 3 x 3:

```
1 2 3
4 5 6
7 8 9
```

padding = 1 后变成 5 x 5:

```
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0
```

使用 3 x 3 卷积核、stride=1 时:

不加 padding: 3 x 3 -> 1 x 1  
加 padding=1: 3 x 3 -> 3 x 3

### 3.6 Stride

页码: p15

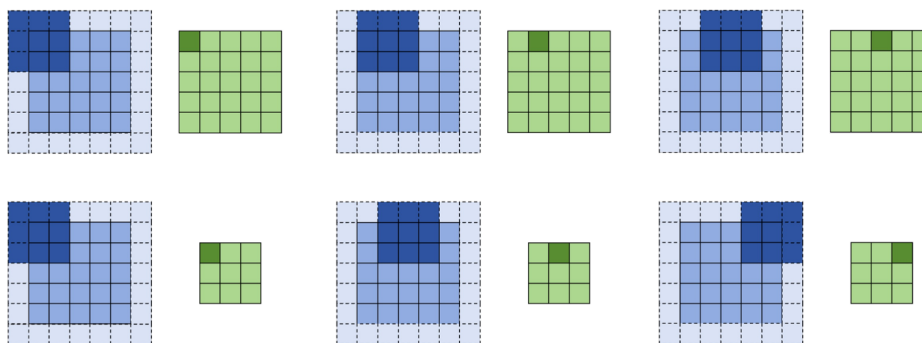
Stride 是卷积核滑动步长。课件用  $\text{stride}(i, j)$  表示横向跨  $i$  步、纵向跨  $j$  步。

**Stride:** Stride 是卷积核每次滑动的步长。stride 越大, 卷积核采样位置越少, 输出空间尺寸通常越小。

#### 3.6.1 Stride 示意

#### 跨步 $\text{stride}(i, j)$

横向跨  $i$  步, 纵向跨  $j$  步。图中第一行为  $\text{stride}(1,1)$ , 第二行为  $\text{stride}(2,2)$



[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



Figure 3: p15 stride(1,1) 和 stride(2,2) 的滑动差别

输入长度方向位置:

```
1 2 3 4 5
```

卷积核大小 = 3

stride = 1 时:

[1 2 3], [2 3 4], [3 4 5]

输出长度 = 3

stride = 2 时:

[1 2 3], [3 4 5]

输出长度 = 2

### 3.7 多通道卷积

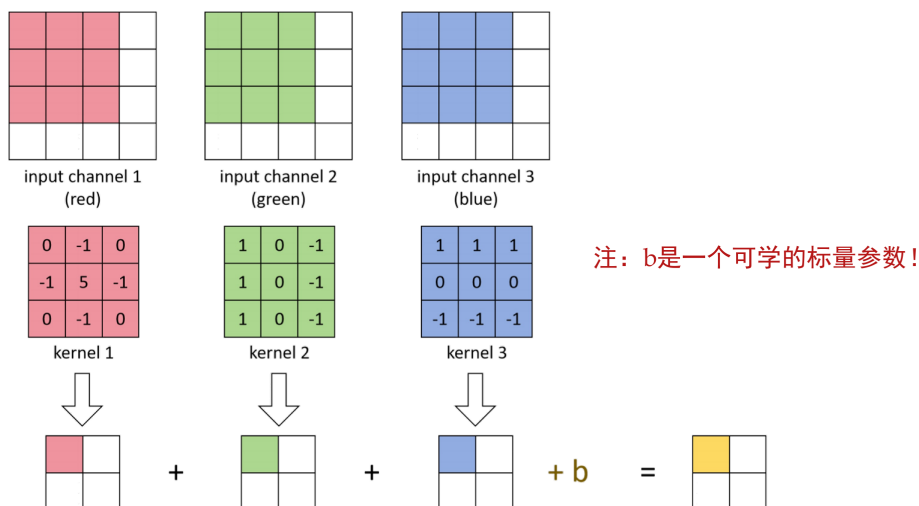
页码: p16

RGB 图像有 3 个输入通道。多通道卷积中，一个完整卷积核需要覆盖所有输入通道。

多通道卷积: 对于有  $C_{in}$  个输入通道的数据，一个完整卷积核的深度也必须覆盖  $C_{in}$  个通道；一个卷积核产生一个输出通道。

#### 3.7.1 多通道卷积示意

### 多通道卷积



[1] 《深度学习现象导论》 许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



Figure 4: p16 多通道卷积示意

RGB 输入图像有 3 个通道:

R 通道局部区域 -> 与卷积核 R 部分相乘求和

G 通道局部区域 -> 与卷积核 G 部分相乘求和

B 通道局部区域 -> 与卷积核 B 部分相乘求和

三个通道结果相加，再加 bias

= 这个卷积核在当前位置的输出

一个卷积核产生一个输出通道:

输入:  $32 \times 32 \times 3$   
一个卷积核大小:  $5 \times 5 \times 3$   
卷积核个数: 6

输出: 空间大小另算, 通道数 = 6

### 3.8 卷积层尺寸计算总结

页码: p14-p16

卷积层的计算可以分成三件事: 空间大小怎么变, 通道数怎么变, 参数量怎么算。

卷积层输出形状: 输入为  $H \times W \times C_{in}$ , 卷积核个数为  $C_{out}$  时, 输出形状为  $H_{out} \times W_{out} \times C_{out}$ ; 输出通道数由卷积核个数决定。

#### 3.8.1 输入、卷积核和输出的形状

输入:  $H \times W \times C_{in}$

一个卷积核:  $K \times K \times C_{in}$   
卷积核个数:  $C_{out}$

输出:  $H_{out} \times W_{out} \times C_{out}$

其中:

- $H, W$ : 输入图像或特征图的高和宽。
- $C_{in}$ : 输入通道数。
- $K$ : 卷积核大小, 例如  $3 \times 3$  卷积核中  $K = 3$ 。
- $C_{out}$ : 卷积核个数, 也就是输出通道数。
- $H_{out}, W_{out}$ : 输出特征图的高和宽。

#### 3.8.2 输出空间尺寸公式

$$H_{out} = \text{floor}((H + 2P - K) / S) + 1$$

$$W_{out} = \text{floor}((W + 2P - K) / S) + 1$$

其中:

- $P$  是 padding 大小。
- $S$  是 stride 大小。
- floor 表示向下取整。

#### 3.8.3 padding 和 stride 的影响

| 参数      | 增大后通常会怎样 | 直观理解             |
|---------|----------|------------------|
| padding | 输出空间尺寸变大 | 边缘补了一圈, 能滑动的位置更多 |
| stride  | 输出空间尺寸变小 | 卷积核跳得更远, 采样位置更少  |
| 卷积核大小   | 输出空间尺寸变小 | 窗口更大, 可放置的位置更少   |
| 卷积核个数   | 输出通道数变多  | 一个卷积核产生一个输出通道    |

#### 3.8.4 参数量计算

卷积层通常有 bias。每个卷积核产生一个输出通道, 因此每个输出通道对应 1 个 bias。

不考虑 bias:

参数量 =  $K \times K \times C_{in} \times C_{out}$

考虑 bias:

参数量 =  $K \times K \times C_{in} \times C_{out} + C_{out}$

例子:

输入通道数  $C_{in} = 3$

卷积核大小  $K = 5$

卷积核个数  $C_{out} = 6$

不考虑 bias:  $5 \times 5 \times 3 \times 6 = 450$

考虑 bias:  $5 \times 5 \times 3 \times 6 + 6 = 456$

### 3.8.5 例子 1: padding 保持大小

输入:  $32 \times 32 \times 3$

卷积核:  $3 \times 3$

卷积核个数: 16

padding: 1

stride: 1

$H_{out} = (32 + 2 \times 1 - 3) / 1 + 1 = 32$

$W_{out} = 32$

$C_{out} = 16$

输出:  $32 \times 32 \times 16$

### 3.8.6 例子 2: stride 让尺寸减小

输入:  $32 \times 32 \times 3$

卷积核:  $3 \times 3$

卷积核个数: 16

padding: 1

stride: 2

$H_{out} = \text{floor}((32 + 2 \times 1 - 3) / 2) + 1$   
 $= \text{floor}(31 / 2) + 1$   
 $= 16$

$W_{out} = 16$

$C_{out} = 16$

输出:  $16 \times 16 \times 16$

## 3.9 池化 Pooling

页码: p18

池化层用于对局部区域做汇总。常见池化包括最大池化和平均池化。

池化: 池化是在局部窗口内做汇总操作, 例如取最大值或平均值, 常用于降低空间尺寸、减少计算量。

| 池化方式 | 操作         |
|------|------------|
| 最大池化 | 取局部窗口中的最大值 |
| 平均池化 | 取局部窗口中的平均值 |

### 3.9.1 池化示意

局部区域:

```
1  3
2  4
```

最大池化: 取最大值 4

平均池化: 取平均值  $(1+3+2+4)/4 = 2.5$

尺寸例子:

输入:  $32 \times 32 \times 16$

最大池化窗口:  $2 \times 2$

stride: 2

输出:  $16 \times 16 \times 16$

### 3.10 CNN 整体结构与层的组合

页码: p20-p22

p20 给出了一个典型 CNN 结构:

输入 RGB 三通道图像

-> 卷积 + ReLU + 池化

-> 卷积 + ReLU + 池化

-> 展平

-> 全连接

-> Softmax 输出类别概率

经典 LeNet 可作为类似结构的例子:

输入图像 -> 卷积 -> 池化 -> 卷积 -> 池化 -> 展平 -> 全连接 -> 输出

需要注意: CNN 没有唯一固定的网络格式。卷积层、激活函数、池化层、全连接层等都可以按任务需要组合。

常见组合方式:

卷积 -> 激活 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 池化 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 展平 -> 全连接 -> 输出

只要前一层的输出形状能作为后一层的输入, 这些层就可以灵活搭配。不同 CNN 结构的差别, 往往就体现在层的数量、顺序、卷积核个数、是否使用池化等设计上。

### 3.11 关键记忆

- 图像具有局部关联性和一定不变性; CNN 的设计正是为了利用这些图像特征。p3-p6
- 指数衰减下降较快, 幂级数衰减下降较慢; 幂级数衰减可以看作许多指数衰减成分的叠加。p3-p5
- 类视皮层设计启发 CNN: 先提取局部简单特征, 再逐步整合成复杂信息。p7-p10
- 卷积的核心是局部窗口加权求和; CNN 相比 MLP 的重要特点是局部连接和参数共享。p11-p13
- Padding 在边缘补值, 用来控制输出空间大小; padding 越大, 输出空间尺寸通常越大。p14
- Stride 是卷积核滑动步长; stride 越大, 输出空间尺寸通常越小。p15
- 多通道卷积中, 一个完整卷积核覆盖所有输入通道; 一个卷积核产生一个输出通道。p16
- 卷积输出空间尺寸看  $H, W, K, P, S$ ; 输出通道数看卷积核个数  $C_{out}$ 。p14-p16
- 卷积参数数量看  $K, C_{in}, C_{out}$ ; 有 bias 时再加  $C_{out}$ , 不要乘  $H_{out}$  和  $W_{out}$ 。p14-p16
- 池化汇总局部信息, 降低空间尺寸, 通常没有可学习参数, 也通常不改变通道数。p18
- CNN 没有唯一固定格式; 卷积、激活、池化、展平、全连接、softmax 等模块可以灵活组合, 但相邻层的输入输出尺寸必须匹配。p20-p22

### 3.12 思考题

1. 判断：池化操作一定只能由专门的池化层实现，不能用卷积实现类似效果。  
答案：错误。例如使用带 stride 的卷积可以在提取特征的同时降低空间尺寸，起到类似下采样的作用。
2. 判断：池化操作可以和卷积操作在同一层的设计中部分合并，例如用 stride 大于 1 的卷积完成下采样。  
答案：正确。
3. 填空：1 x 1 卷积核不改变单个位置的空间邻域大小，但可以改变 \_\_\_\_\_ 数。  
答案：通道。
4. 计算：输入为 32 x 32 x 3，卷积核为 3 x 3，padding=1，stride=1，卷积核个数为 16，输出尺寸是 \_\_\_\_\_。  
答案：32 x 32 x 16。
5. 计算：一个卷积层中 K=5，C\_in=3，C\_out=6，且每个卷积核有 bias，参数量是 \_\_\_\_\_。  
答案：5 x 5 x 3 x 6 + 6 = 456。

## 4 循环神经网络 RNN 与 LSTM

这一章按 PDF 顺序复习：语言任务特点 -> 数据准备 -> 词元化 -> one-hot 与 embedding -> next token prediction -> RNN 设计 -> BPTT -> LSTM。重点是让学生理解：文本是可变长度序列，RNN 为什么适合处理序列，以及 LSTM 的门控机制解决了什么问题。

### 4.1 自然语言处理 NLP 的任务特点

页码：p3-p5

NLP 的目标是让计算机理解、生成和处理自然语言。常见任务包括机器翻译、文本分类、语音识别、信息检索、智能客服等。

自然语言有层次结构、语法结构、上下文相关性和长程依赖。更重要的是，文本天然是可变长度的离散符号序列。

自然语言处理 NLP：NLP 是让计算机理解、生成和处理人类自然语言的技术方向。

#### 4.1.1 文本数据的可变长度

文本和图像的一个关键区别是：文本序列长度通常不固定。不同句子的 token 数可能不同：

我喜欢 AI  
我非常喜欢人工智能这门课  
这部电影虽然节奏很慢，但是后半段非常精彩

这和图像任务很不一样。图像通常可以统一 resize 成固定大小，例如 32 x 32 x 3；文本句子却天然长短不一，并且词语顺序会影响含义。

语言数据的几个特点：

| 特点     | 含义              | 例子                |
|--------|-----------------|-------------------|
| 离散符号序列 | 文本由词、子词、字符等符号组成 | “我 / 喜欢 / AI”     |
| 顺序重要   | 词语顺序会影响含义       | “我打他”和“他打我”       |
| 上下文相关  | 当前词含义依赖上下文      | “上海的交通大学”和“上海的交通” |
| 长程依赖   | 较远位置的信息可能影响后面理解 | 前文主语影响后文指代        |

#### 4.1.2 MLP、CNN 与 RNN 的区别

| 模型  | 输入特点         | 是否天然适合可变长度序列 |
|-----|--------------|--------------|
| MLP | 常需要固定长度向量    | 不适合          |
| CNN | 擅长局部窗口特征     | 不天然适合        |
| RNN | 逐 token 处理序列 | 适合           |

RNN 能处理可变长度文本的原因是：它不是一次性要求输入固定长度向量，而是按时间步逐个读入 token，并把已经读过的信息压缩到隐藏状态中。

**RNN 处理可变长度序列的核心：**RNN 按时间步逐个处理输入 token，并在每一步更新隐藏状态；同一个 RNN 单元可以重复使用任意多次，因此可以处理不同长度的序列。

短句：x1 -> x2 -> x3

长句：x1 -> x2 -> x3 -> x4 -> x5 -> ...

同一个 RNN 单元可以在不同时间步重复使用。

## 4.2 数据准备

页码：p6-p7

数据准备是语言模型训练的基础。考试不考复杂数据工程流程，但要知道文本数据进入模型前通常需要清洗和处理。

| 步骤     | 作用                     |
|--------|------------------------|
| 数据收集   | 获得足够数量的文本              |
| 质量过滤   | 去掉质量很差、无意义或噪声很大的文本     |
| 敏感内容过滤 | 减少有毒内容、隐私信息等风险         |
| 数据去重   | 减少重复样本，避免模型过度记忆重复文本    |
| 词元化    | 把文本切成 token            |
| 向量化    | 把 token 变成神经网络能处理的数值向量 |

关键理解：数据不是直接进入 RNN。文本通常要先经过清洗、切分和向量化，才能变成神经网络输入。

## 4.3 分词、one-hot 与 embedding

页码：p8-p11

文本进入神经网络通常要经过两步：先把文本切成 token，再把 token 变成向量。

文本 -> tokenization -> token 序列 -> one-hot 或 embedding 向量

### 4.3.1 Tokenization 词元化

Tokenization 是把文本切成 token。Token 可以是词、子词、字符或特殊符号。

**Tokenization：**Tokenization 是把原始文本切分成 token 序列的过程；输入是文本，输出是 token 序列。

词元化可以基于规则，也可以基于统计。考试重点不是背具体分词算法，而是知道“文本如何变成 token 序列”。

原始句子：

我喜欢人工智能

词级 tokenization：

[我，喜欢，人工智能]

子词级 tokenization：

[我，喜欢，人工，智能]

特殊符号不要求逐个背，但要知道它们也是 token。比如 PAD 常用于把不同长度句子补齐到同一长度，UNK 常用于表示词表中没有出现过的词。

### 4.3.2 one-hot 与 embedding

One-hot 用一个很长的向量表示词表中的一个 token，向量中只有一个位置是 1，其余位置是 0。

Embedding 把 token 映射成低维稠密向量。Embedding 矩阵是可训练参数，可以在训练中学习词语之间的关系。

**One-hot:** One-hot 是用词表长度的稀疏向量表示一个 token，只有该 token 对应位置为 1，其余位置为 0。

**Embedding:** Embedding 是把 token 映射成低维稠密向量的表示方法，通常由可训练的 embedding 矩阵得到。

| 表示方式      | 特点        | 记忆       |
|-----------|-----------|----------|
| one-hot   | 高维、稀疏     | 不表达词义相似性 |
| embedding | 低维、稠密、可训练 | 可以学习语义关系 |

例子:

词表: [我, 喜欢, 人工智能, 这门课]

token = 喜欢

one-hot = [0, 1, 0, 0] # 维度 = 词表大小 4

embedding = [0.12, -0.38, 0.51] # 维度 = 设定的 embedding 维度 3

Embedding 的计算可以理解为查表: 用 token 的 one-hot 向量从 embedding 矩阵中取出对应的低维向量。

## 4.4 Next Token Prediction

页码: p13-p16

Next Token Prediction 是根据已有前文预测下一个 token。它是现代语言模型常用的训练框架。

**Next Token Prediction:** 给定前文 token 序列，模型预测下一个 token；训练时每个位置的目标通常是输入序列中下一个位置的 token。

## Next Token Prediction框架

语言模型的训练普遍采用Next Token Prediction框架，即输出序列的每一个token为输入序列的下一个 token



假设输入为:  $X^{\text{in}} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$

模型的预测输出为:  $\hat{X}^{\text{out}} = (\hat{\mathbf{x}}_2, \hat{\mathbf{x}}_3, \dots, \hat{\mathbf{x}}_{n+1})$

根据Next Token Prediction, 我们期望它的输出为:  $X^{\text{out}} = (\mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_{n+1})$

于是可以对其计算交叉熵损失:

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=2}^{n+1} \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$

输入：我 喜欢 人工  
目标：智能

大量文本都可以转化为“预测下一个 token”的任务。它的好处是：不需要人工为每句话单独标标签，只要有文本，就可以构造训练信号。

训练：每个位置预测下一个 token

推理：生成一个 token 后，把它接到上下文后继续生成

#### 4.4.1 Next Token Prediction 的 loss

模型输出的是下一个 token 的概率分布。训练时常用交叉熵损失：真实 token 的概率越高，loss 越小。

单个位置的交叉熵：

Loss =  $-\log P(\text{真实下一个 token} \mid \text{前文})$

例子：

前文：我 喜欢 人工

真实下一个 token：智能

如果模型给“智能”的概率是 0.8：

Loss =  $-\log(0.8)$

如果模型给“智能”的概率是 0.1：

Loss =  $-\log(0.1)$

第二种 loss 更大，因为模型把真实答案的概率预测得太低。

## 4.5 RNN 的设计

页码：p17-p18

RNN 是循环神经网络，适合处理序列数据。它的核心设计是：当前隐藏状态不仅看当前输入，也看上一时刻隐藏状态。

## 循环神经网络 (Recurrent Neural Networks)

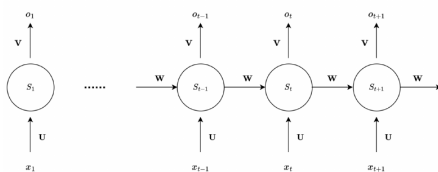


图 3.19: 循环神经网络的基本结构

如图3.19所示，这是一个基本的循环神经网络单元，下面我们详细介绍一下这个单元是如何工作的。我们的输入为时序序列  $x_1, x_2, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_n$ ，并通过公式(3.15)、(3.16)和(3.17)的方式计算得到对应的输出序列  $o_1, o_2, \dots, o_{t-1}, o_t, o_{t+1}, \dots, o_n$ 。

$$S_0 = 0, \quad (3.15)$$

$$S_t = f(\mathbf{U} \cdot x_t + \mathbf{W} \cdot S_{t-1} + b), \quad t = 1, 2, 3, \dots, n, \quad (3.16)$$

$$o_t = g(\mathbf{V} \cdot S_t) \quad (3.17)$$

其中  $f$  是激活函数，通常使用 Tanh 函数， $g = \text{Softmax}(x)$  是 softmax 函数。 $S_t$  是状态记忆， $\mathbf{U}$ ， $\mathbf{W}$ ， $\mathbf{V}$  和  $b$  分别是输入权重矩阵、循环权重矩阵、输出权重矩阵和偏置项。

**隐藏状态：**RNN 的隐藏状态  $S_t$  是模型在第  $t$  步保存的历史信息摘要，它由当前输入  $x_t$  和上一时刻隐藏状态  $S_{t-1}$  共同决定。

RNN 的隐藏状态可以理解为“目前为止读过内容的压缩记忆”。序列长度不同也没关系，因为 RNN 可以重复使用同一个计算单元，按时间步逐个处理输入。

考试要会识别 RNN 的基本公式：

$$\begin{aligned} S_0 &= 0 \\ S_t &= f(U x_t + W S_{t-1} + b) \\ o_t &= g(V S_t) \end{aligned}$$

其中：

- $x_t$ ：第  $t$  个输入。
- $S_t$ ：第  $t$  步隐藏状态。
- $S_{t-1}$ ：上一时刻隐藏状态。
- $o_t$ ：第  $t$  步输出。
- $U, W, V, b$ ：可训练参数。
- $f$  常用  $\tanh$ ， $g$  可以是  $\text{softmax}$ 。

这一段的重点只记三件事：

1.  $S_t$  同时依赖当前输入  $x_t$  和历史状态  $S_{t-1}$ 。
2. 参数  $U, W, V$  在不同时间步共享。
3. 输入多少个 token，就重复多少次同样的 RNN 单元，所以可以处理可变长度序列。

Encoder-Decoder RNN 可以处理输入输出长度不同的任务，例如机器翻译。

## 4.6 BPTT 与梯度问题

页码：p20

BPTT 是 BackPropagation Through Time，意思是随时间反向传播。训练 RNN 时，误差要沿多个时间步向前传播回去。

**BPTT：**BPTT 是随时间反向传播，把 RNN 按时间步展开后，沿多个时间步计算梯度。

普通反向传播：沿网络层反向传播

BPTT：沿时间展开后的网络反向传播

长序列中，梯度需要经过很多次链式相乘，容易出现两个问题：

| 问题   | 含义     | 影响           |
|------|--------|--------------|
| 梯度消失 | 梯度越来越小 | 较早时间步的信息难以学习 |
| 梯度爆炸 | 梯度越来越大 | 训练不稳定        |

因此，普通 RNN 对长程依赖的建模能力有限。

BPTT 面临的问题和解决思路：

| 问题   | 为什么出现             | 解决思路                  |
|------|-------------------|-----------------------|
| 梯度消失 | 长序列中梯度多次链式相乘后变得很小 | LSTM 用记忆状态和门控机制缓解长程依赖 |
| 梯度爆炸 | 长序列中梯度多次链式相乘后变得很大 | 工程中常用梯度裁剪；本章重点理解 LSTM |

## 4.7 LSTM

页码: p21-p25

LSTM 是 Long Short-Term Memory, 长短期记忆网络。它在 RNN 基础上引入记忆状态, 并通过门控机制控制信息流动。

### 长短期记忆网络 (Long short-term memory, LSTM)

- 基本原则: 1) 任何信息使用前先做线性变换, 权重可学  
2) 控制比例的称为“门”, 激活用 Sigmoid  
3) 提取信息的运算, 激活用 Tanh

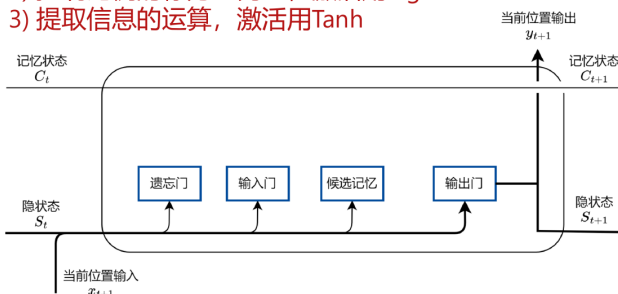


图 3.21: 单个门控神经元主要部件和并行隐藏状态示意图

为什么能称为“遗忘门”、“输入门”、“输出门”、“知识”？  
这是一种形象类比, 没有严格证明。

#### 门控神经元-遗忘门

保留的比例  $f_t = \sigma(W_{Sf}S_t + W_{xf}x_{t+1} + b_f)$

#### 门控神经元-输入门

写入的比例  $i_t = \sigma(W_{Si}S_t + W_{xi}x_{t+1} + b_i)$

新的知识  $\tilde{C}_{t+1} = \tanh(W_{SC}S_t + W_{xC}x_{t+1} + b_C)$

新的记忆状态  $C_{t+1} = f_t * C_t + i_t * \tilde{C}_{t+1}$

#### 门控神经元-输出门

输出的比例  $o_t = \sigma(W_{So}S_t + W_{xo}x_{t+1} + b_o)$

输出  $y_{t+1} = S_{t+1} = o_t * \tanh(C_{t+1})$



**LSTM:** LSTM 是一种改进的 RNN, 它通过记忆状态和门控机制控制信息的保留、写入和输出, 用来缓解普通 RNN 的长程依赖问题。

| 结构   | 作用         |
|------|------------|
| 记忆状态 | 保存较长期的信息   |
| 遗忘门  | 控制保留多少旧记忆  |
| 输入门  | 控制写入多少新信息  |
| 输出门  | 控制输出多少记忆信息 |

门通常使用 sigmoid, 因为 sigmoid 输出在 0 到 1 之间, 适合表示“保留多少”“写入多少”这种比例。候选记忆常用 tanh 提取。

门控机制: 门控机制用 0 到 1 之间的比例控制信息流动: 遗忘门控制旧记忆保留多少, 输入门控制新信息写入多少, 输出门控制当前输出多少记忆信息。

LSTM 设计里最重要的点:

1. 多了一条记忆状态  $C_t$ , 用于保存较长期信息。
2. 用门控制比例, 而不是简单地全保留或全丢弃。
3. 遗忘门决定旧记忆保留多少。
4. 输入门决定新信息写入多少。
5. 输出门决定当前输出多少记忆信息。

LSTM 不能保证完全消除所有梯度问题, 但它可以缓解普通 RNN 的长程依赖和梯度消失问题。

## 4.8 关键记忆

- NLP 处理的是离散符号序列，词语顺序和上下文都很重要。
- 文本数据通常是可变长度的；MLP/CNN 不天然适合记忆历史信息，RNN 通过隐藏状态逐步处理序列。
- 文本进入神经网络前通常要经过数据清洗、tokenization 和向量化。
- Tokenization 是把文本切成 token；token 可以是词、子词、字符或特殊符号。
- One-hot 高维稀疏，本身不表达词义相似性；embedding 低维稠密，可以通过训练学习语义关系。
- Next Token Prediction 是根据前文预测下一个 token，是现代语言模型常用训练框架。
- RNN 的当前隐藏状态依赖当前输入和上一时刻隐藏状态，公式为  $S_t = f(U x_t + W S_{t-1} + b)$ 。
- BPTT 是随时间反向传播；长序列中容易出现梯度消失或梯度爆炸。
- LSTM 通过记忆状态和门控机制控制信息保留、写入和输出，可以缓解长程依赖问题。

## 4.9 思考题

1. 判断：文本数据通常是可变长度序列，因此普通 MLP 不天然适合直接处理任意长度文本。  
答案：正确。
2. 填空：把文本切成 token 的过程叫做 \_\_\_\_\_。  
答案：tokenization 或词元化。
3. 填空：RNN 的基本公式中，当前隐藏状态可以写成  $S_t = f(U x_t + W \_\_\_\_\_\_ + b)$ 。  
答案： $S_{t-1}$ 。
4. 判断：RNN 可以处理可变长度序列，一个重要原因是同一个 RNN 单元可以在不同时间步重复使用。  
答案：正确。
5. 选择：LSTM 中控制保留多少旧记忆的是：A. 输入门 B. 遗忘门 C. 输出门 D. Softmax。  
答案：B。

# 5 Transformer

这一章按“为什么需要 Transformer -> embedding 与位置编码 -> 注意力机制 -> Q/K/V -> causal mask -> Transformer block 与输出”的顺序复习。重点是理解 Transformer 如何用注意力机制建模 token 之间的关系。

## 5.1 为什么需要 Transformer

Transformer 出现前，序列任务常用 RNN 和 LSTM。RNN/LSTM 能处理序列，但有两个明显限制：

| 问题      | 含义               |
|---------|------------------|
| 长程依赖困难  | 很远位置的信息不容易传到当前时刻 |
| 并行计算不方便 | RNN 通常按时间步顺序计算   |

Transformer 使用注意力机制直接建模 token 之间的关系，更容易捕捉全局语义关系，也更适合并行计算。

## 5.2 Embedding 与位置编码

Token 是离散符号，进入神经网络前需要先变成向量。

token -> embedding 向量

但仅有 embedding 还不够。同一个 token 出现在不同位置时，句子含义可能不同，因此模型还需要位置信息。

输入表示 = token embedding + 位置编码

位置编码用于告诉模型 token 在序列中的位置。位置编码可以是可学习的，也可以是人为设计的。

## 5.3 注意力机制直观理解

注意力机制可以分成两步：

第一步：算“我应该关注谁”

第二步：按关注程度汇总信息

对某个 token 来说，模型会先计算它和其他 token 的关联强度，再把这些关联强度变成权重，对其他 token 的信息加权求和。

| 概念      | 含义            |
|---------|---------------|
| 注意力分数   | token 之间的关联强度 |
| Softmax | 把分数变成权重       |
| 注意力权重   | 通常非负，总和为 1    |
| 加权求和    | 按关注程度汇总信息     |

注意力权重越大，说明该位置对当前 token 的更新影响越大。

## 5.4 Q、K、V

注意力机制中常见三个向量：Q、K、V。

| 符号 | 英文    | 作用              |
|----|-------|-----------------|
| Q  | Query | 当前 token 发出的查询  |
| K  | Key   | 其他 token 可被匹配的键 |
| V  | Value | 真正被汇总的信息内容      |

Q、K、V 都由输入向量经过可训练线性变换得到。Q 和 K 用来计算相关性，V 用来做加权求和。

Q 和 K → 算关联强度

Softmax → 得到注意力权重

注意力权重 和 V → 加权求和

## 5.5 Causal Mask

在生成任务中，模型预测当前位置时不能看到未来 token，否则就等于提前看到了答案。

已知：我 喜欢

预测：人工

不能偷看：智能

Decoder 使用 causal mask 遮住未来位置。常见做法是把未来位置的注意力分数设为负无穷，经过 Softmax 后，这些位置的权重接近 0。

Causal mask 保证 next token prediction 的因果性：预测当前位置时只能看当前位置和前文，不能看未来。

## 5.6 Transformer Block、FNN 与输出

一个 Transformer block 通常包含注意力模块和前馈网络 FNN，并配合残差连接和 LayerNorm。

输入

→ 注意力模块

→ 残差连接 + LayerNorm

→ FNN

→ 残差连接 + LayerNorm

→ 输出

FNN 是前馈全连接网络，对每个 token 分别作用。多个 Transformer block 可以复制堆叠，形成深层 Transformer。

输出层把隐藏向量映射到词表大小，再经过 Softmax 得到下一个 token 的概率分布。

| 解码方式 | 含义             | 结果是否固定 |
|------|----------------|--------|
| 贪婪解码 | 每步取概率最大的 token | 通常固定   |
| 采样解码 | 按概率分布随机抽样      | 可能不同   |

## 5.7 关键记忆

- Transformer 的核心思想之一是注意力机制，用来建模 token 之间的关系。
- Transformer 仍然需要位置信息；位置编码用于表示 token 在序列中的位置。
- 注意力机制先计算关联强度，再用 Softmax 得到权重，最后对信息加权求和。
- Q 和 K 用于计算相关性，V 是真正被汇总的信息内容。
- Decoder 做 next token prediction 时不能看到未来 token，需要 causal mask。
- Transformer block 通常由注意力模块、FNN、残差连接和 LayerNorm 等部分组成。
- 输出层维度通常对应词表大小；Softmax 把输出转成下一个 token 的概率分布。
- 采样解码具有随机性，同一输入可能生成不同结果。

## 5.8 思考题

1. 判断：Transformer 使用注意力机制来建模 token 之间的关系。  
答案：正确。
2. 填空：Transformer 需要加入 \_\_\_\_\_，用来表示 token 在序列中的位置。  
答案：位置编码。
3. 选择：Q、K、V 中，用来真正汇总信息内容的是：A. Q B. K C. V D. mask。  
答案：C。
4. 判断：Decoder 做 next token prediction 时，可以看到未来 token。  
答案：错误。需要 causal mask 遮住未来位置。
5. 判断：采样解码按概率分布随机抽样，因此同一输入可能生成不同结果。  
答案：正确。

# 6 神经网络训练现象：频率原则

这一章不适合考复杂推导，重点考概念和直觉。

## 6.1 过参数化与隐式偏好

复习重点：

1. 过参数化指模型参数量大于样本量或明显很大。
2. 过参数化模型理论上容易表达很多复杂函数。
3. 但神经网络实际训练中常常仍能泛化较好。
4. 训练误差为 0 的解可能有很多。
5. 神经网络训练过程会倾向某些解，这叫隐式偏好。

易错点：

1. “过参数化神经网络一定严重过拟合。”错误。
2. “训练过程可能偏向某类解。”正确。
3. “隐式偏好一定写在损失函数公式里。”错误。

## 6.2 频率、低频与高频

复习重点：

1. 频率描述函数变化的快慢。
2. 低频对应平滑、轮廓、整体趋势。
3. 高频对应振荡、细节、快速变化。
4. 图像中低频常对应大轮廓，高频常对应边缘、纹理或噪声。

5. 高频不一定没用，但更容易包含噪声。

易错点：

1. “低频通常更平滑。”正确。
2. “高频永远没有用。”错误。
3. “细节和噪声都可能表现为高频。”正确。

### 6.3 频率原则

复习重点：

1. 频率原则也叫 spectral bias。
2. 神经网络训练通常先拟合低频，再拟合高频。
3. 在一维函数中表现为先学平滑轮廓，再学振荡细节。
4. 在图像拟合中表现为先学整体轮廓，再学局部细节。
5. 频率原则可以帮助解释为什么一些神经网络不容易立即拟合噪声。

必背句：

频率原则：神经网络在训练过程中通常按照从低频到高频的顺序拟合目标函数。

易错点：

1. “频率原则说神经网络通常先学高频噪声。”错误。
2. “频率原则和泛化有关系。”正确。
3. “频率原则只适用于分类任务，不能用于函数拟合直觉。”错误。

### 6.4 早停 Early stopping

复习重点：

1. 数据中可能有噪声。
2. 低频成分通常先被模型学习。
3. 高频噪声往往在训练后期被逐渐拟合。
4. 早停是在模型过度拟合噪声前停止训练。
5. 早停有助于提高测试集泛化能力。

易错点：

1. “训练越久，测试误差一定越低。”错误。
2. “早停可能防止拟合高频噪声。”正确。
3. “早停就是让模型完全不训练。”错误。

### 6.5 思考题

1. 填空：频率原则也叫 spectral \_\_\_\_。  
答案：bias。
2. 判断：神经网络训练通常先拟合低频成分，再拟合高频成分。  
答案：正确。
3. 选择：图像中的边缘、纹理、噪声通常更接近：A. 低频 B. 高频 C. 标签 D. bias。  
答案：B。
4. 判断：过参数化神经网络一定会严重过拟合，因此不可能泛化好。  
答案：错误。
5. 判断：早停可以理解为在模型进一步拟合高频噪声前停止训练。  
答案：正确。

## 7 参数凝聚与解的平坦性

### 7.1 初始化大小的影响

复习重点：

1. 初始化是训练开始前给参数赋初值。
2. 初始化会影响训练过程和最终结果。
3. 初始化太大时，网络输出可能更随机、更复杂，更容易过拟合。
4. 初始化较小时，网络输出可能更平滑。
5. 常见初始化包括 LeCun、Xavier、Kaiming。
6. 模型越大时，初始化尺度通常需要更谨慎。
7. 所有参数不能简单初始化为 0，因为会破坏训练中的对称性打破。

易错点：

1. “初始化对训练结果没有任何影响。”错误。
2. “所有参数初始化为 0 是最安全的做法。”错误。
3. “初始化大小会影响训练。”正确。

### 7.2 参数凝聚

复习重点：

1. 参数凝聚指小初始化下，同层神经元会有趋同的偏好。
2. 多个相似神经元可以等效为较少的有效神经元。
3. 凝聚使大网络在表达上像复杂度更低的小网络。
4. 过参数化大网络发生凝聚后，仍然比直接训练小网络更容易优化。
5. 凝聚可理解为一种复杂度控制。

必背句：

参数凝聚：小初始化下，同层神经元会出现趋同的偏好，多个神经元可能等效为较少的有效神经元。

易错点：

1. “参数凝聚通常与小初始化有关。”正确。
2. “凝聚意味着网络完全不能训练。”错误。
3. “凝聚可以看作降低有效复杂度的一种现象。”正确。

### 7.3 平坦解与尖锐解

复习重点：

1. 平坦解指参数附近一片区域损失都较低。
2. 尖锐解指参数稍微改变，损失就明显上升。
3. 训练集和测试集常有噪声，平坦解对扰动更鲁棒。
4. 平坦解通常泛化更好。
5. 小批量训练倾向于找到更平坦的解。
6. 合理较大的学习率也可能帮助找到更平坦的解。
7. Dropout 也可能有助于提升解的平坦性。

易错点：

1. “平坦解对参数扰动更鲁棒。”正确。
2. “尖锐解一定比平坦解泛化更好。”错误。
3. “小批量训练可能帮助找到更平坦的解。”正确。

### 7.4 思考题

1. 判断：初始化大小会影响神经网络的训练过程和最终结果。  
答案：正确。

- 判断：所有参数初始化为 0 通常不是好做法，因为会破坏对称性打破。  
答案：正确。
- 填空：小初始化下，同层神经元可能出现趋同偏好，这种现象叫 \_\_\_\_\_。  
答案：参数凝聚。
- 选择：平坦解通常指：A. 参数附近一片区域损失都较低 B. 参数必须全为 0 C. 训练误差一定为 1 D. 没有梯度。  
答案：A。
- 判断：尖锐解对参数扰动更敏感，泛化通常不如平坦解稳健。  
答案：正确。

## 8 大模型介绍

### 8.1 什么是大模型

复习重点：

- 大模型通常也叫基础模型。
- 大模型参数量很大。
- 大模型经过大规模数据训练。
- 大模型可以适应多种下游任务。
- 大语言模型是最早展现出通用智能特征的大模型之一。
- 许多模态的数据也可以类似语言一样进行 token 化。

易错点：

- “大模型通常经过大规模数据训练。”正确。
- “大模型只能做一个固定任务。”错误。
- “基础模型可以适应多种下游任务。”正确。

### 8.2 Encoder、Decoder 与常见架构

复习重点：

- Encoder 可以看到上下文所有 token。
- Decoder 只能看到当前位置和前文 token。
- Encoder-only 常用于理解任务。
- Decoder-only 常用于生成任务。
- Encoder-Decoder 常用于输入和输出都需要建模的任务，如翻译。
- BERT 是 Encoder-only 的代表。
- GPT、Llama、DeepSeek 等是 Decoder-only 的代表。
- BART、T5 是 Encoder-Decoder 的代表。
- 当前主流生成式大语言模型多采用 Decoder-only。

对照表：

| 架构              | 是否看未来     | 常见任务 | 代表                 |
|-----------------|-----------|------|--------------------|
| Encoder-only    | 可以看双向上下文  | 理解任务 | BERT               |
| Decoder-only    | 只能看当前和前文  | 生成任务 | GPT/Llama/DeepSeek |
| Encoder-Decoder | 编码输入，解码输出 | 翻译等  | T5/BART            |

易错点：

- “Decoder-only 在生成任务中很常见。”正确。
- “Encoder-only 非常适合直接做 next token prediction。”通常不适合。
- “BERT 是 Decoder-only。”错误。

### 8.3 预训练、微调、提示词、蒸馏

复习重点：

1. 预训练是在大规模数据上训练基础能力。
2. 语言模型预训练常用 next token prediction。
3. 微调是在预训练模型基础上，用特定任务数据继续训练。
4. 强化学习后训练可增强推理能力，也可用于对齐人类偏好。
5. 提示词工程通过设计输入让模型给出更好的回答。
6. 知识蒸馏把大模型能力迁移到小模型中。
7. Teacher model 是提供知识的大模型或强模型。
8. Student model 是接受蒸馏的小模型或目标模型。

易错点：

1. “预训练通常需要大量数据。”正确。
2. “提示词工程一定会更新模型参数。”错误。
3. “知识蒸馏可以让小模型学习大模型输出中的知识。”正确。

### 8.4 大模型中的现象

复习重点：

1. Scaling law 描述模型规模、数据量、计算量和测试损失之间的规律关系。
2. 一般来说，在一定范围内，规模、数据、算力增加会带来性能提升。
3. 上下文学习指模型不更新参数，仅通过任务描述和示例适应任务。
4. 思维链是让模型生成中间推理步骤。
5. 思维链可提升复杂推理任务表现。
6. 幻觉是模型生成看似真实但实际错误或误导的信息。
7. 大模型可能包含偏见。

易错点：

1. “上下文学习不需要更新模型权重。”正确。
2. “思维链就是引导模型写出中间推理步骤。”正确。
3. “大模型永远不会产生幻觉。”错误。
4. “大模型可能带有偏见。”正确。

### 8.5 思考题

1. 判断：大模型通常经过大规模数据训练，可以适应多种下游任务。  
答案：正确。
2. 选择：当前主流生成式大语言模型多采用：A. Encoder-only B. Decoder-only C. 纯 CNN D. 纯 RNN。  
答案：B。
3. 填空：预训练是在大规模数据上训练模型的 \_\_\_\_\_ 能力。  
答案：基础。
4. 判断：上下文学习通常不更新模型参数，而是通过任务描述和示例让模型适应任务。  
答案：正确。
5. 判断：幻觉指模型生成看似真实但实际错误或误导的信息。  
答案：正确。

## 9 强化学习

这一章是考试重点，但不建议考复杂推导。重点是概念、关系和最简单公式。

### 9.1 强化学习是什么

复习重点：

1. 强化学习是决策型任务。
2. 智能体在环境中采取动作。
3. 环境返回奖励和新的状态。
4. 智能体通过反复试错学习。
5. 目标是最大化期望累计奖励。
6. 奖励可以是正的，也可以是负的。
7. 强化学习不同于监督学习：通常没有每一步的正确标签，而是通过奖励反馈学习。

基本循环：

状态  $s_t$   $\rightarrow$  智能体选择动作  $a_t$   $\rightarrow$  环境给出奖励  $r_{t+1}$  和下一状态  $s_{t+1}$

易错点：

1. “强化学习只做静态分类任务。”错误。
2. “强化学习强调智能体和环境交互。”正确。
3. “奖励只能是正数。”错误。

## 9.2 状态、观察、动作空间、奖励

复习重点：

1. 状态 State 是环境完整信息。
2. 观察 Observation 是智能体实际看到的信息。
3. 完全可观测环境中，观察和状态一致。
4. 部分可观测环境中，观察只是状态的一部分。
5. 动作空间是所有可选动作的集合。
6. 动作空间可以是离散的，也可以是连续的。
7. 棋类、超级马里奥常可看作离散动作空间。
8. 自动驾驶中的方向盘角度、油门等常可看作连续动作空间。
9. 奖励是评价动作好坏的反馈信号。

易错点：

1. “观察一定等于完整状态。”错误。
2. “动作空间可以是离散的，也可以是连续的。”正确。
3. “奖励用于告诉智能体行为好坏。”正确。

## 9.3 回报与折扣因子

复习重点：

1. 强化学习不只关心当前奖励，还关心未来奖励。
2. 回报是未来奖励的累积。
3. 折扣因子  $\gamma$  用于降低远期奖励权重。
4.  $\gamma$  的取值范围是  $0 \leq \gamma \leq 1$ 。
5.  $\gamma$  越接近 0，越重视眼前奖励。
6.  $\gamma$  越接近 1，越重视长期奖励。

无折扣回报：

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

例题：

未来三个奖励分别为：1, 0, 2

$\gamma = 0.5$

$$G_t = 1 + 0.5 \times 0 + 0.5^2 \times 2$$

$$= 1 + 0 + 0.25 \times 2$$

$$= 1.5$$

易错点：

1. “强化学习完全不关心未来奖励。”错误。
2. “折扣因子通常在 0 到 1 之间。”正确。
3. “gamma 越接近 1，越重视长期奖励。”正确。

## 9.4 情节型任务与连续型任务

复习重点：

1. 情节型任务有起点和终点。
2. 一局游戏、一次棋局、一个关卡通常是情节型任务。
3. 连续型任务没有固定终止状态。
4. 股票交易、交通控制、自动驾驶可看作连续型任务。

易错点：

1. “所有强化学习任务都有明确终点。”错误。
2. “棋类游戏通常可以看作情节型任务。”正确。
3. “连续型任务需要持续决策。”正确。

## 9.5 探索与利用

复习重点：

1. 探索 Exploration 是尝试未知或不确定动作以获取信息。
2. 利用 Exploitation 是根据已有知识选择当前看起来最好的动作。
3. 只利用可能错过更好的策略。
4. 只探索可能无法获得足够高奖励。
5. 强化学习需要平衡探索和利用。

生活例子：

每天去熟悉但还不错的餐厅：利用  
尝试一家没去过的新餐厅：探索

易错点：

1. “探索是尝试未知动作。”正确。
2. “利用是根据已有知识做当前较优选择。”正确。
3. “强化学习中完全不需要探索。”错误。

## 9.6 策略、价值函数与最优策略

复习重点：

1. 策略 policy 是智能体选择动作的规则。
2. 策略可以是确定性的，也可以是概率性的。
3. 价值函数  $V(s)$  衡量从状态  $s$  出发能获得的期望回报。
4. 状态价值高，说明从这个状态出发更有希望获得高累计奖励。
5. 最优策略是在长期意义下使期望累计奖励最大的策略。
6. 最优价值函数和最优策略相互关联。

易错点：

1. “策略就是状态到动作选择的规则。”正确。
2. “价值函数只看当前一步奖励，完全不看未来。”错误。
3. “最优策略追求最大化期望累计奖励。”正确。

## 9.7 MDP 与马尔可夫性

复习重点：

1. MDP 是 Markov Decision Process，马尔可夫决策过程。
2. MDP 是强化学习常用数学模型。
3. 马尔可夫性指未来只依赖当前状态和动作，而不依赖完整历史。
4. 如果当前状态已经包含足够信息，就不需要再记住所有过去信息。
5. MDP 常包含状态、动作、转移概率、奖励、折扣因子等元素。

必背句：

马尔可夫性：给定当前状态和动作后，未来状态的概率分布与更早历史无关。

易错点：

1. “MDP 是马尔可夫决策过程。”正确。
2. “马尔可夫性要求未来依赖全部历史。”错误。
3. “当前状态如果足够完整，就可以用于预测下一状态分布。”正确。

## 9.8 Bellman 方程的直观含义

复习重点：

1. Bellman 方程刻画当前价值和未来价值的递推关系。
2. 当前状态的价值等于当前奖励加上折扣后的下一状态价值的期望。
3. Bellman 方程是许多强化学习算法的基础。
4. 不要求推导复杂公式，只要理解“当前价值 = 当前收益 + 未来价值”。

直观形式：

当前价值 = 当前奖励 +  $\gamma$  × 未来价值

不考虑动作时，向量形式：

$$V = R + \gamma P V$$

易错点：

1. “Bellman 方程把当前价值和未来价值联系起来。”正确。
2. “Bellman 方程完全不考虑未来。”错误。
3. “Bellman 方程是强化学习中的重要基础。”正确。

## 9.9 DP、MC 与深度强化学习

复习重点：

1. 动态规划 DP 可以利用 Bellman 方程迭代求价值。
2. DP 的优点是可以较精确计算价值。
3. DP 的缺点是通常要知道环境转移概率  $P$ 。
4. 蒙特卡洛 MC 通过多次实验采样取平均。
5. MC 的优点是简单直接，不一定需要明确知道转移概率。
6. MC 的缺点是随机性大，方差可能大。
7. 深度强化学习把深度神经网络引入强化学习。
8. 深度神经网络可以近似价值函数或策略。
9. DQN 用神经网络替代表格来拟合  $Q$  值。
10. 深度强化学习能处理高维状态，如图像输入。
11. 深度强化学习也带来训练不稳定、数据需求大、算力需求高等问题。

对照表：

| 方法  | 核心思想           | 优点       | 缺点           |
|-----|----------------|----------|--------------|
| DP  | 用 Bellman 方程迭代 | 可较精确计算   | 需要环境模型/转移概率  |
| MC  | 多次采样取平均        | 简单直接     | 随机性大，方差可能大   |
| DRL | 用深度网络近似价值或策略   | 处理高维复杂状态 | 训练难、数据和算力需求大 |

易错点：

1. “DP 通常需要知道转移概率。”正确。
2. “MC 通过采样估计价值。”正确。
3. “DQN 用神经网络近似 Q 值。”正确。
4. “深度强化学习完全没有训练困难。”错误。

## 9.10 强化学习应用

复习重点：

1. 强化学习可用于游戏 AI。
2. 强化学习可用于机器人控制。
3. 强化学习可用于自动驾驶、交通灯调度、推荐系统、数据中心节能等。
4. 大模型后训练中也可使用强化学习。
5. 在 ChatGPT 一类系统中，可以用人类偏好构造奖励信号。
6. 在推理模型中，强化学习可用于增强推理能力。

易错点：

1. “强化学习只能玩游戏。”错误。
2. “强化学习可以用于机器人控制。”正确。
3. “大模型后训练中可能使用强化学习。”正确。

## 9.11 思考题

1. 判断：强化学习中，智能体通过与环境交互，并根据奖励反馈学习。  
答案：正确。
2. 填空：强化学习的目标通常是最大化期望 \_\_\_\_\_ 奖励。  
答案：累计。
3. 计算：未来三个奖励为 1, 0, 2,  $\gamma = 0.5$ , 折扣回报为 \_\_\_\_\_。  
答案： $1 + 0.5 \times 0 + 0.5^2 \times 2 = 1.5$ 。
4. 选择：尝试一个不确定但可能更好的动作，属于：A. 探索 B. 利用 C. 监督学习 D. 归一化。  
答案：A。
5. 判断：马尔可夫性指给定当前状态和动作后，未来状态分布不再依赖更早历史。  
答案：正确。

# 10 最后一遍串讲清单

## 10.1 残差网络必会

1. 深层普通网络可能出现退化问题。
2. 退化问题指网络加深后训练误差或测试误差反而变差。
3. 深层网络还可能遇到梯度消失和梯度爆炸。
4. 残差连接的基本形式是  $F(x) + x$ 。
5. 残差连接帮助信息和梯度在深层网络中传播。
6. ResNet 主要用于缓解深层网络训练困难和退化问题。

## 10.2 CNN 必会

1. 图像具有局部关联性和一定不变性。
2. 关联性会随距离衰减，课件中有指数衰减和幂级数衰减两类模型。
3. 幂级数衰减可以理解许多指数衰减成分的叠加，长程关联更不容易完全消失。
4. CNN 与 MLP 的核心区别是局部连接和参数共享。
5. 卷积核在图像上滑动，用局部窗口提取边缘、方向、纹理等局部特征。
6. 卷积核参数通常随机初始化，然后训练得到。
7. RGB 输入有 3 个通道；输出通道数由卷积核个数决定。
8. 会根据输入大小、卷积核大小、padding、stride 计算输出空间尺寸；输出通道数等于卷积核个数。
9. 会根据卷积核大小、输入通道数、卷积核个数计算卷积层参数量。
10. 池化可以降低空间尺寸，逐渐处理更长范围的关联，通常不改变通道数。
11. CNN 可由卷积、激活、池化、展平、全连接、输出层等模块按任务需要组合。

## 10.3 RNN/LSTM 必会

1. NLP 是处理自然语言的任务。
2. 语言有顺序、上下文、长程依赖。
3. Tokenization 把文本切分成 token。
4. One-hot 稀疏且不表达语义相似。
5. Embedding 可训练，能学习词向量。
6. Next Token Prediction 是预测下一个 token。
7. RNN 能处理序列，因为有历史状态。
8. BPTT 训练 RNN，长序列可能梯度消失/爆炸。
9. LSTM 通过记忆状态和门控机制缓解长程依赖问题。

## 10.4 Transformer 必会

1. Transformer 核心是注意力机制。
2. Embedding 把 token 变向量。
3. 位置编码提供顺序信息。
4. Q/K/V 分别是 Query/Key/Value。
5. Q 和 K 算相关性，V 被加权求和。
6. Softmax 把注意力分数变成权重。
7. Causal mask 防止 Decoder 看未来。
8. LayerNorm 逐 token 归一化。
9. FNN 逐 token 作用。
10. 输出层映射到词表大小。

## 10.5 训练现象必会

1. 频率原则：先低频后高频。
2. 低频是轮廓、平滑、整体趋势。
3. 高频是细节、振荡、快速变化，也可能包含噪声。
4. 早停可以减少拟合高频噪声。
5. 初始化影响训练。
6. 小初始化可能导致参数凝聚。
7. 参数凝聚使多个神经元趋向相似偏好。
8. 平坦解通常更鲁棒、泛化更好。

## 10.6 大模型必会

1. 大模型又叫基础模型。
2. 大模型通常参数量大、数据规模大、适应任务多。
3. Encoder-only 常用于理解任务。
4. Decoder-only 常用于生成任务。
5. 当前主流生成式大语言模型多是 Decoder-only。

6. 预训练学习基础能力。
7. 微调用特定任务数据继续训练。
8. 提示词工程改变输入，不一定改参数。
9. 知识蒸馏把大模型知识迁移到小模型。
10. 上下文学习不更新权重。
11. 思维链生成中间推理步骤。
12. 大模型可能产生幻觉和偏见。

## 10.7 强化学习必会

1. 强化学习是决策型任务。
2. 基本循环：状态、动作、奖励、下一状态。
3. 目标是最大化期望累计奖励。
4. 状态是完整环境信息，观察是智能体看到的信息。
5. 动作空间可以离散或连续。
6. 折扣因子  $\gamma$  在 0 到 1 之间。
7.  $\gamma$  越大越重视长期奖励。
8. 情节型任务有终点，连续型任务没有固定终点。
9. 探索是尝试未知，利用是使用已有知识。
10. 策略是选择动作的规则。
11. 价值函数衡量状态的期望回报。
12. MDP 是马尔可夫决策过程。
13. 马尔可夫性：未来只依赖当前状态和动作。
14. Bellman 方程联系当前价值和未来价值。
15. DP 需要环境模型，MC 通过采样估计。
16. 深度强化学习用神经网络近似价值函数或策略。

## 11 不建议作为考试重点的内容

以下内容可以课堂提到，但不建议作为选择、判断、填空题重点。

1. 卷积平移等变性的严格证明。
2. Pooling 反向传播细节。
3. LSTM 门控公式完整推导。
4. 正余弦位置编码、RoPE、相对位置编码的公式推导。
5. 多头注意力复杂矩阵维度计算。
6. 多尺度神经网络、Fourier 特征、NeRF。
7. Hessian、稳定边缘的数学推导。
8. LoRA、MCP、Agent、RAG 的技术细节。
9. TD、Sarsa、Q-learning 的公式推导。
10. Kaggle 注册、实验提交、代码复现操作。

## 12 两节复习课建议讲法

### 12.1 第一节课：网络结构主线

建议顺序：

1. 5 分钟：说明考试范围和“不考阅读材料”。
2. 10 分钟：残差网络，重点讲退化问题、梯度传播困难和残差连接。
3. 30 分钟：CNN，按“图像数据特点 -> 为什么需要卷积 -> 卷积/padding/stride/池化计算 -> 经典结构”讲。
4. 20 分钟：RNN/LSTM，重点讲序列、BPTT、梯度问题、门控机制。
5. 25 分钟：Transformer，重点讲 embedding、位置编码、QKV、mask、输出层。

课堂建议：

1. CNN 部分一定要带学生完整算 2-3 个尺寸题。

2. Transformer 部分不要推矩阵公式，重点讲 Q/K/V 的角色。
3. RNN/LSTM 部分不要推门控公式，重点讲为什么需要记忆和门。

## 12.2 第二节课：训练现象、大模型、强化学习

建议顺序：

1. 20 分钟：频率原则、早停、初始化、凝聚、平坦解。
2. 20 分钟：大模型架构、预训练、微调、提示词、蒸馏、scaling law、上下文学习、思维链。
3. 35 分钟：强化学习基本概念、回报计算、探索利用、MDP、Bellman、DP/MC/DRL。
4. 15 分钟：串讲易错点。

课堂建议：

1. 训练现象部分用“先学轮廓再学细节”来讲频率原则。
2. 大模型部分重点做概念辨析，不要展开最新技术细节。
3. 强化学习部分要反复画“状态-动作-奖励-下一状态”的循环。
4. Bellman 方程只讲“当前价值 = 当前奖励 + 折扣未来价值”。